CST 238 Spring 2026

Homework 2. Classes Using Arrays

Due: Thursday, Feb 5, 11:55 pm

How to turn in:

- Write programs and submit them on the course website
- The names are in the assignment; incorrect names will be graded as 0
- Remember to add the header for each file, as it was required in the previous assignment

Part A. Quiz Averages with Classes (15 points)

Develop a program to determine the average of quiz scores of all students in a course. The input data of the course will be stored in an input file, and a record of each student will include the following

- The student's name
- The student's ID
- Five quiz scores (use an array of doubles)
- An average of the students' quiz scores

The file will have up to (but no more than) 30 records, and will have a special string (`STOP`) to indicate EOF (the end of the file). An example file is available below, but you should make your own test files, too.

You must create a class called `Student` to store each student's record. You do not need to store an array of `Student` objects, but it can be helpful. The `Student` class should:

1. Have attributes to store the student's name, ID, and five quiz scores
2. Have a constructor that takes the name and ID as parameters
3. Have a method, `public void setScore(double[] score)`, that stores the given quiz scores (passed in as an array)
4. Have a method, `public double getAverage()`, to get the average score of the top four grades
5. Have a method, `public String toString()`, that will return a string representation of the object as described below

The `main` method in the `Student` class should:

- Ask for an input filename and open the file for reading
- Create a `Student` object for each record (row) in the input
- Print the results for each student to the screen, per the format below

```
Student (ID#): Average
```

Sample Input File (t1.txt):

```
Tom    1000  9.5  9.0  8.5  8.0  8.5
Alice  2000 10.0  6.7 10.0 10.0 10.0
John   3000  6.9  8.0  8.0  8.0  8.0
Alice  1500  8.0  9.0  7.0  6.5  8.0
Jason  2500  5.0  5.0  5.0  3.5  5.0
STOP   -1   -1.0 -1.0 -1.0 -1.0 -1.0
```

Sample Run (user input in **bold**):

```
Enter input filename: t1.txt
-----------------------------------------------
  Course Report: Quiz Average
-----------------------------------------------
  Tom (1000): 8.875
  Alice (2000): 10.0
  John (3000): 8.0
  Alice (1500): 8.0
  Jason (2500): 5.0
-----------------------------------------------
```

Final Notes:

- Read the sample run carefully to understand the requirements of the problem correctly
- This program should be uploaded to the course website as `Student.java`

Part B. Combination Lock (15 pts)

Write a program to simulate the operation of a rotating combination lock (numbers 1 to 40) that gives hints of guessing higher or lower.

Create a class called `Lock` that has these attributes:

- An array of three integers to store the secret numbers (`codes`)
- An integer to store which code the user is currently guessing (`currentCode`)
- An array of up to 120 integers to store all of the guesses (`allGuesses`)
- An integer to store how many total guesses the user has made (`guessCount`)

And the following methods:

- A constructor that
    - Takes three integer codes
    - Initializes the two arrays
    - Copies the values from the parameters into the `codes` array
    - Sets `currentCode` and `guessCount` to zero
- An `isLocked()` method that returns true if the value of the `currentCode` variable is NOT 3
- A `guess()` method that takes an integer guess from the user and
    - Adds it to the `allGuesses` array and increments the `guessCount` variable
    - Checks if it matches the current code
    - If so, print "Correct", increments the `currentCode` variable
    - If not, tells the user which direction move (up or down)
- A `guesses()` method that returns a string of all of the user guesses (with no additional zeros)
- A `toString()` that will return a string containing the codes (for debugging purposes)

```
Enter guess: 0
```

```
Enter guess: 4
Go down
Enter guess: 35
Go down
Enter guess: 10
Go down
Enter guess: 1
Correct!
Enter guess: 28
Go up
Enter guess: 30
Go down
Enter guess: 29
Correct!
Enter guess: 35
Go down
Enter guess: 25
Go up
Enter guess: 30
Correct!
The codes are: 1 29 30
10 guesses: 4 35 10 1 28 30 29 35 25 30
```

Here is the code for the `main` method in the `Lock` class. For testing purposes, you may not want to generate random numbers.

```
public static void main(String[] args) {
  Random r = new Random();
  Scanner in = new Scanner(System.in);
  Lock l = new Lock(r.nextInt(40) + 1,
      r.nextInt(40) + 1,
      r.nextInt(40) + 1);

  while(l.isLocked()) {
      int guess;
      do {
        System.out.print("Enter guess: ");
        guess = in.nextInt();
      } while(guess < 1 || guess > 40);
      l.guess(guess);
  }

  System.out.println(l);
  System.out.println(l.guesses());
}
```

Final Notes:

- Read the sample run carefully to understand the requirements of the problem correctly
- This program should be uploaded to the course website as `Lock.java`